KIRO DEMO SUBMISSION

PromptStudio
提示词工作台

集成式 Prompt / Skill 编辑、多模型对话与可视化修改对比应用
让每一次 AI 改写都可对比、可解释、可回滚

在线体验	https://zawsstt.github.io/prompt-studio/
开源仓库	https://github.com/zawsstt/prompt-studio
提交日期	2026-10-22

核心能力

规则引擎实时校验 · 多模型流式对话
四色 Diff 对比 · Why/How/Effect 修改日志
版本时间线回滚 · 8 家模型一键接入

要解决的问题：AI 改 Prompt，改了什么、为什么改、能不能退？

今天大多数人调 Prompt 的方式是「复制到对话框 → 让模型改写 → 整段覆盖粘贴回来」。这个过程有三个硬伤：**看不见改了什么**（整段覆盖，无从核对）、**不知道为什么改**（只有结果没有理由）、**退不回去**（历史版本靠手工另存）。PromptStudio 把「AI 改写」从一次黑盒替换，变成一个可审阅、可逐块决策、可回滚的工程流程。

① 改了什么?

左右分屏 Diff，行级 LCS + 字符级细化，四色标记新增 / 删除 / 替换 / 重排，支持逐块跳转。

② 为什么改?

每处变更绑定 Why / How / Effect 三段说明，说明可编辑，把 AI 产物变成用户资产。

③ 能不能退?

每次接受修改自动生成版本快照，时间线可视化，一键回滚且不覆盖历史。

产品定位

面向 Prompt 工程师 / Agent 开发者的「带信任机制的修改引擎」：编辑、校验、对话、对比、留痕、回滚，在一个工作台里闭环。

形态

纯前端 Web 应用，零后端依赖；模型 API Key 只存本机（系统凭据库 / 本地加密），浏览器直连厂商端点，无中间代理。

六大核心模块

1 • 编辑器 + 规则引擎

Markdown / YAML frontmatter 编辑，CodeMirror 6 高亮；编辑防抖 500ms 实时校验，行内波浪线 + 右侧问题面板双通道；一键「自动整理」按规则集重排模块。

2 • 多模型对话

8 家供应商流式对话，Reasoning 折叠展示；自动注入当前文档、规则集摘要与迭代摘要；全文 / 增量两种上下文模式。

3 • Diff 对比视图

左右分屏同步滚动，四色变更标记，字符级差异细化；变更摘要面板 + 悬停气泡；键盘 N/P 跳转、A 接受、R 拒绝。

4 • 修改日志

毛玻璃滑入面板，逐块展示 Why / How / Effect、行号范围、片段对照与模型元信息；说明可编辑、可钉住。

5 • 版本时间线

接受修改自动快照，v1→v2→v3 时间线；回滚生成新版本不覆盖历史；可导出变更日志与 SKILL.md。

6 • 模型接入

四步接入：选供应商 → 选模型 → 填 Key → 测连接；内置 8 家模板，支持自定义 OpenAI 兼容端点（内网网关）。

规则集：三类校验

结构 必含/禁止字段、模块顺序　格式 YAML 合法性、代码块语言标签、标题层级、行宽　风格 人称统一、祈使句、禁用模糊词

工作区范围：三层隔离

范围 按 ## 标题自动解析模块　勾选 只把选定模块送进上下文　锁定 锁定模块只读，防止 AI 越界改动

使用流程：从导入文档到定稿留痕

STEP 01
导入 / 新建
拖入 .md 文件或整个文件夹，自动解析 frontmatter 与 ## 模块。

→

STEP 02
选规则集
Anthropic Skill 标准 / 通用 Prompt / OpenAI Function / 自定义。

→

STEP 03
实时校验
结构 / 格式 / 风格三类规则即时报错，点击定位到行。

→

STEP 04
对话改写
选模型，写清修改意图，流式拿到候选文档。

→

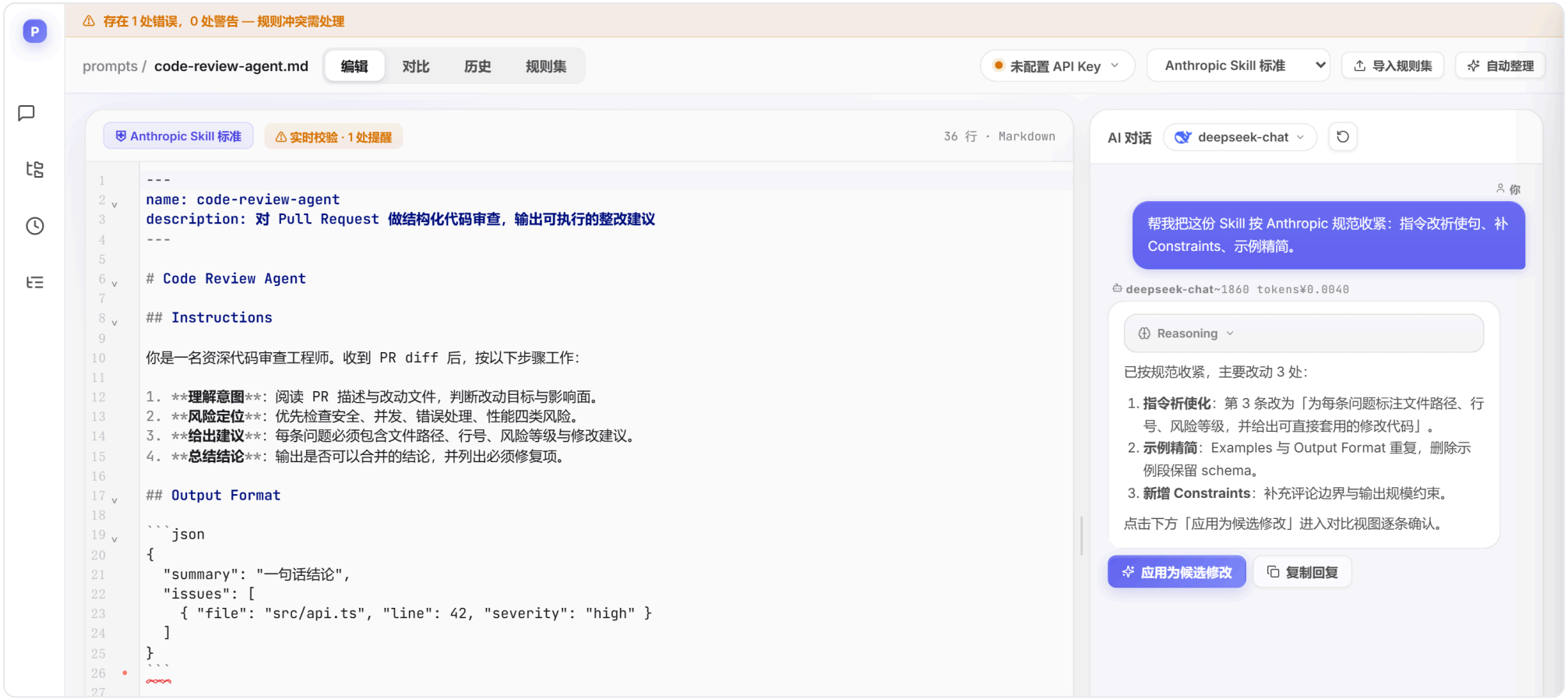
STEP 05
逐块审阅
Diff 对比 + 修改日志，逐条接受/拒绝，全部处理完生成新版本。

上下文可控
全文模式发送整篇 + 规则集 + 迭代摘要；增量模式只发送选中区块，节省 40%+ token。

决策可逆
每处变更独立决策，接受/拒绝实时反映到行内状态；退出前提醒未处理项。

结果可带走
定稿后可导出 SKILL.md 与 Markdown 变更日志，直接进代码仓库。

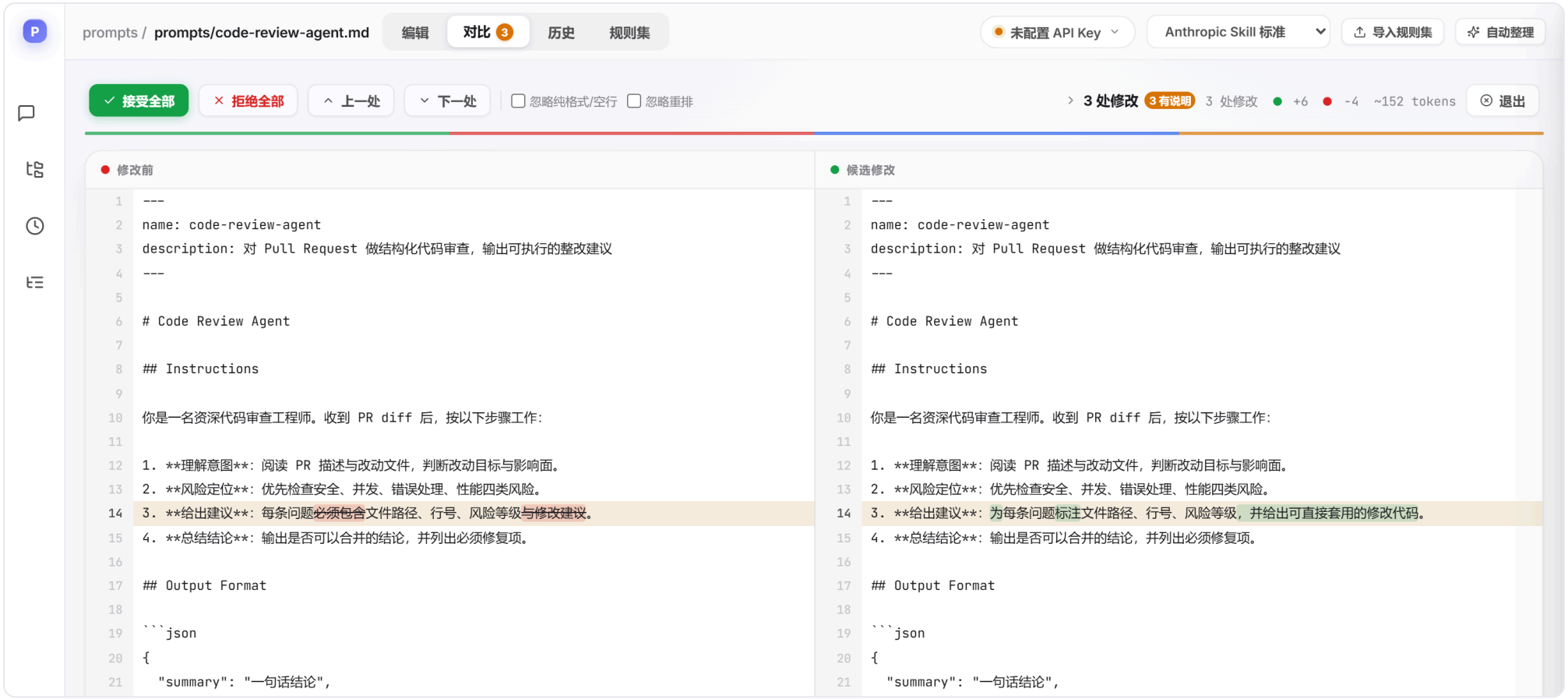
界面演示 1 • 三栏工作台：编辑、校验、对话同屏



字幕解说
00:05 – 00:35

左栏是 **CodeMirror 编辑器**，打开的是 **code-review-agent.md**；右侧实时校验面板报出 1 处格式错误——第 26 行的代码块缺少语言标签，问题可直接点击定位。中间顶部的下拉是**规则集切换**，当前使用「Anthropic Skill 标准」。右栏是 **AI 对话面板**，本轮把用户的三点要求整理成三条改动说明，Reasoning 区块折叠保留模型的思考过程；底部「应用为候选修改」是把回复转成候选文档的唯一入口——**AI 永远不能直接改文档。**

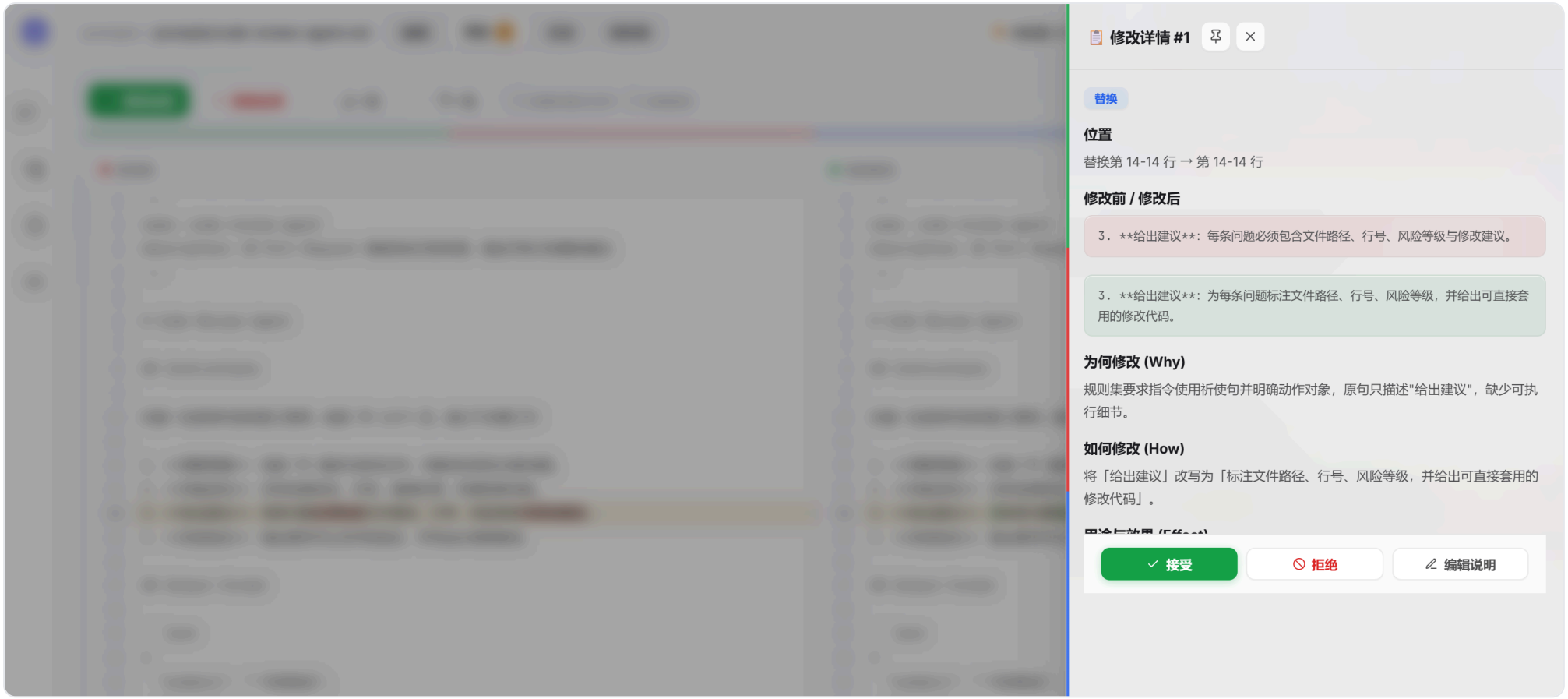
界面演示 2 • Diff 对比：四色标记，逐块决策



字幕解说
00:36 – 01:10

点击「应用为候选修改」后进入对比视图：左侧修改前、右侧候选修改，**滚动完全同步**。本页共 **3 处修改**：绿色是新增的 Constraints 章节，黄色是替换（第 3 条指令改为祈使句），红色是删除（重复的 Examples 段），蓝色表示重排。顶部工具条给出统计 **+6 / −4 / ~152 tokens**，并可一键接受全部或拒绝全部；「忽略格式/空行」「忽略重排」两个开关让对比只看内容变化。

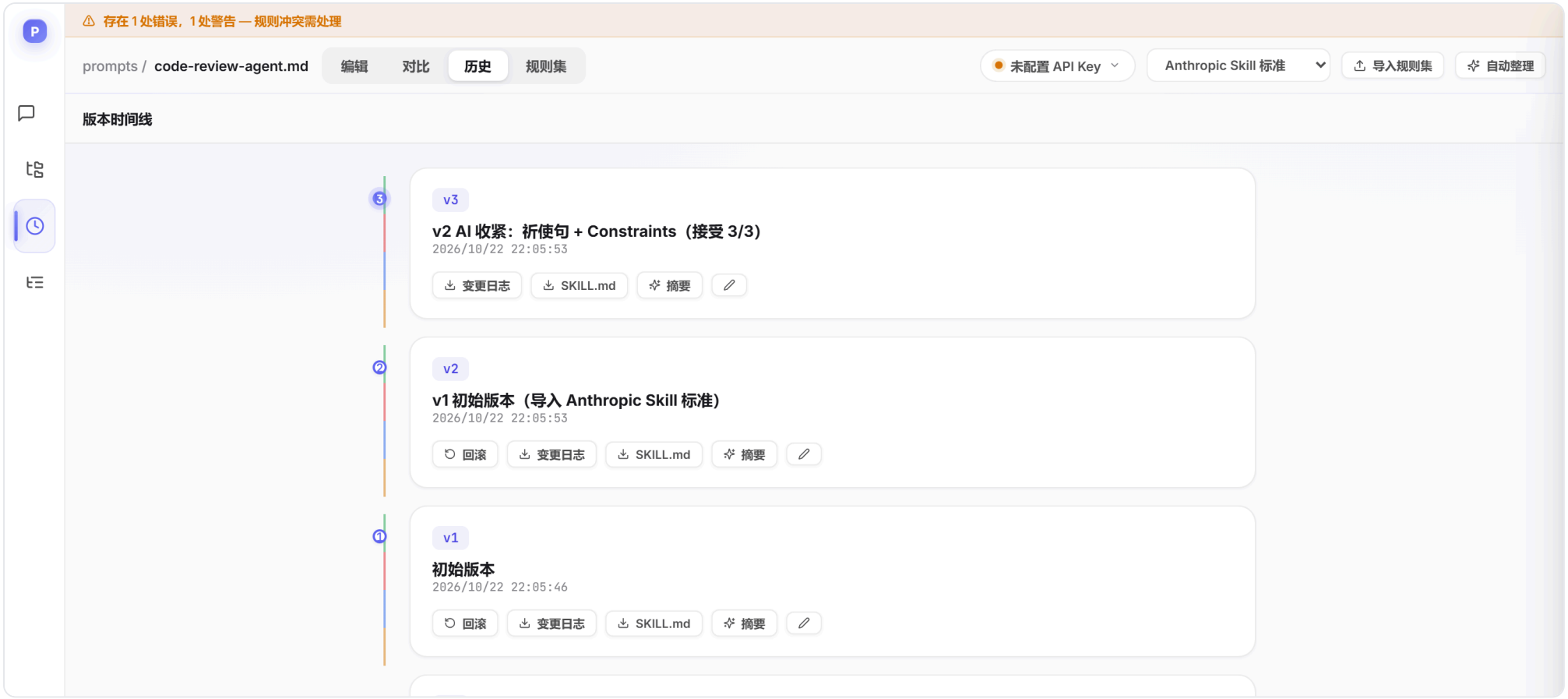
界面演示 3 • 修改日志：每处改动都有 Why / How / Effect



🎙 字幕解说
01:11 – 01:45

点击任一变更块，右侧滑出**毛玻璃修改日志面板**。它把这一处修改拆成三段：**Why** 为什么改（触发了哪条规则）、**How** 怎么改（具体手法）、**Effect** 改完有什么效果（对指令遵循率、token 成本的影响）。下方还记录行号范围、模型名、token 与成本估算。说明文本**用户可以自己改写**——这样修改日志就不只是 AI 的产物，而是团队的评审记录。

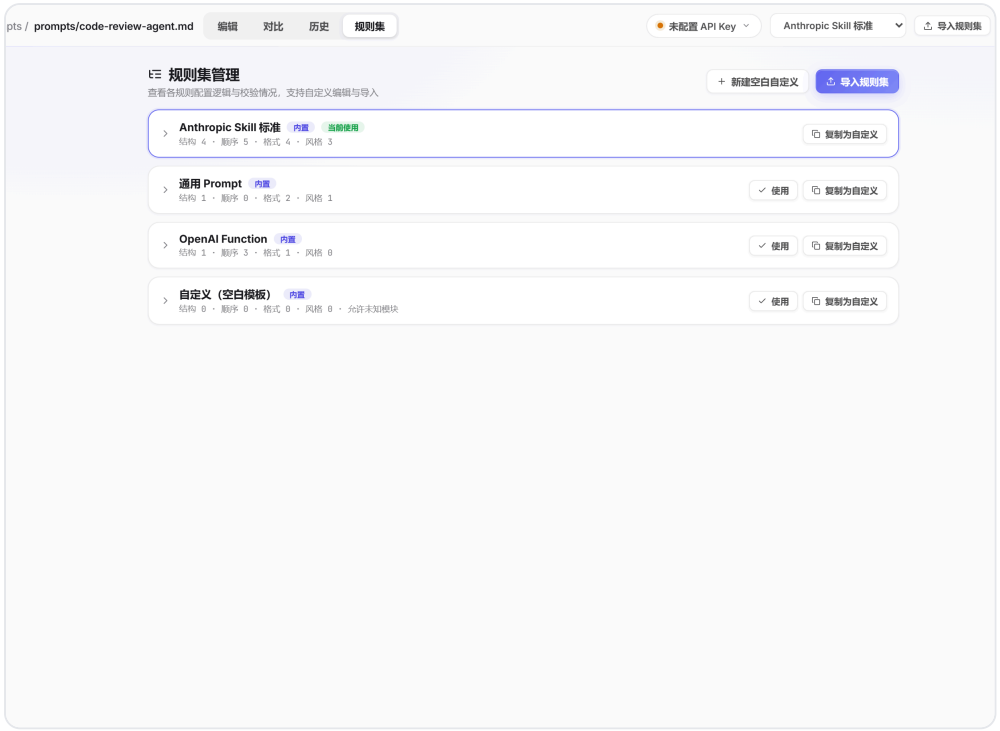
界面演示 4 • 版本时间线：每一步都可回溯

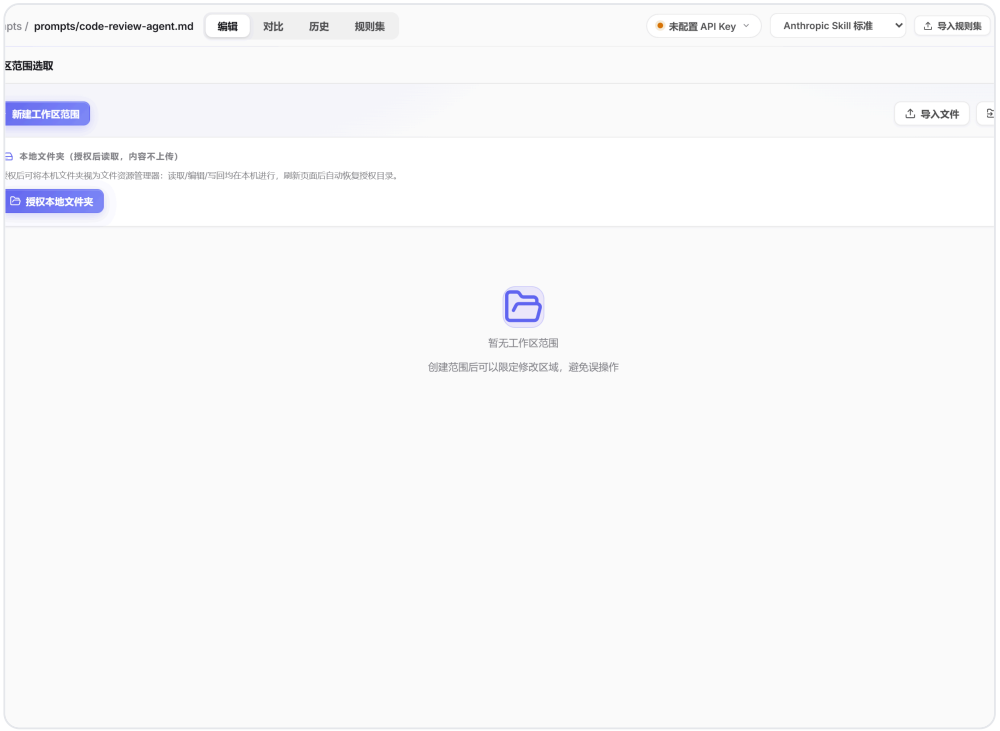


字幕解说 01:46 – 02:10

所有变更处理完毕后生成新版本快照，在「历史」里以**时间线**呈现：每个节点标注版本号、备注、时间与变更统计。回滚不会覆盖历史，而是**以旧版本内容生成一个新版本**，所以任何一次「反悔」本身也被记录。每个版本都可以导出**变更日志**（Markdown）和 **SKILL.md**，直接进仓库或交给团队复用。

界面演示 5 · 规则集与工作区范围：把规范变成可执行的检查

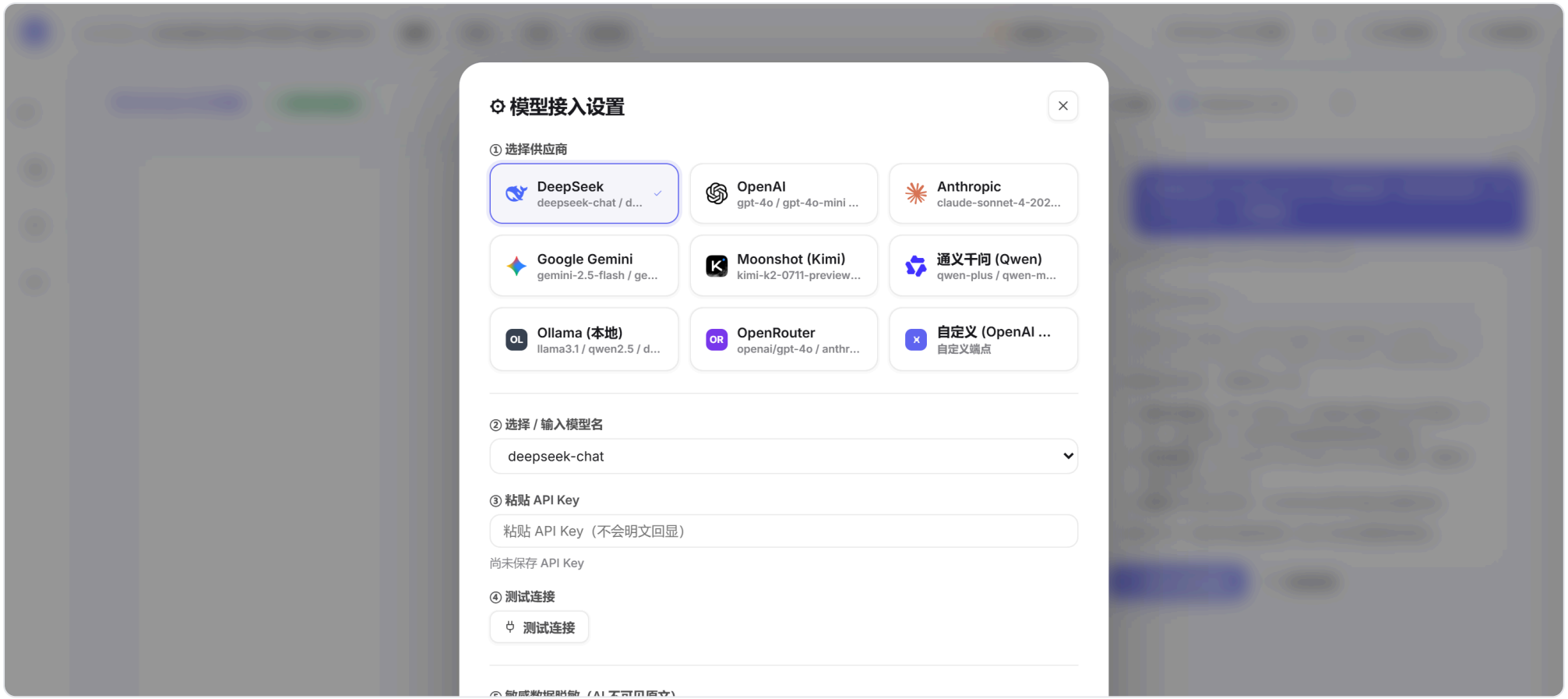




字幕解说
02:11 – 02:40

左图是**规则集管理**：结构、排序、格式、风格四类规则可逐条查看和调整，也可以把团队现成的规范 .md **导入并提炼成规则集**。右图是**工作区范围**：按 ## 标题自动解析出模块，勾选哪些模块参与 AI 上下文、哪些**锁定为只读**，让 AI 只在授权区域内改动。

界面演示 6 • 模型接入：四步接入，Key 不出本机



🎙 字幕解说
02:41 – 03:05

接入流程只有四步：**选供应商 → 选/填模型名 → 粘贴 API Key → 测试连接。**内置 OpenAI、DeepSeek、Anthropic、Gemini、通义千问、Moonshot、Ollama、OpenRouter 八家模板，也可自定义 OpenAI 兼容端点接公司内网网关。API Key **只存本机**（系统凭据库优先，其次本地 AES-256-GCM 加密），输入后不可回显，浏览器直连厂商端点，**没有任何中间代理。**

技术实现方式

前端框架	TypeScript + React 18，Vite 构建，纯静态产物部署 GitHub Pages
编辑器	CodeMirror 6（markdown / yaml 语法高亮、行号、选区捕获）
状态管理	Zustand 单一 Store，集中管理文档、对话、Diff、版本四类状态
Diff 引擎	自研行级 LCS 算法 + diff-match-patch 字符级细化，输出结构化 ChangeBlock[]
规则引擎	结构 / 排序 / 格式 / 风格四类规则声明式配置，逐行扫描产出带行号的 issue 列表
流式对话	原生 fetch + ReadableStream 解析 SSE，AbortController 支持随时中断
本地持久化	IndexedDB（Dexie）存文档 / 对话 / 版本，localStorage 存设置
密钥安全	系统凭据库（Windows Credential Manager / macOS Keychain）优先，回退本地 AES-256-GCM

三个关键设计决策

1

AI 只产出候选，不直接落盘

：模型回复必须经「应用为候选修改」转成候选文档，再由用户在 Diff 中逐块接受，才写回正文。

2

变更块是唯一数据单位

：Diff 的 ChangeBlock 同时承载变更内容、说明、接受状态与模型元信息，修改日志、版本快照、变更日志导出全部复用它。

3

零后端

：没有服务端存储与代理，密钥与文档都不离开浏览器，部署成本为零，也避免了「把 Prompt 传给第三方」的信任问题。

使用了哪些 Kiro 能力

① Spec 驱动开发

先用 Kiro 的 Spec 模式把想法写成需求 → 设计 → 任务三件套（对应仓库里的 PRD、design-proposal、模块功能说明），再让 Agent 按任务清单逐条实现，避免一次性生成一大坨不可维护的代码。

② Agentic 多文件编码

编辑器、Diff、修改日志、版本、设置等模块由 Agent 跨文件生成与重构，包括 Store action、组件与样式联动改动，人工只做验收与关键决策。

③ Steering 规范约束

把设计规范（色板、动效时长、缓动曲线、组件约定）写进 steering 文件，让后续每次生成都自动遵守同一套设计语言，不用反复在对话里重复交代。

④ Hooks 自动校验

配置保存/构建时自动跑类型检查与脚本验证，把「改完就跑一遍」固化成钩子，减少回归问题。

⑤ 终端与浏览器预览闭环

在 Kiro 内直接起 Vite 开发服务器、跑 Puppeteer 截图脚本核对界面，形成「改代码 → 看真实页面 → 再改」的快速迭代回路。

⑥ 上下文引用与对话式调试

用 #File / #Folder 把相关源码、设计稿、报错信息拉进上下文，让 Agent 定位跨模块问题（如 Diff 与修改日志的绑定关系）。

资源清单与复现方式

在线链接

Demo 体验：https://zawsstt.github.io/prompt-studio/

开源仓库：https://github.com/zawsstt/prompt-studio

GitHub Pages 自动部署（push main 触发）

本地文件下载路径

项目根目录：D:\pp studio

本说明文档：D:\pp studio\submission\PromptStudio-Demo成果说明.pdf

界面截图：D:\pp studio\docs\demo-shots\

需求与设计：PRD-PromptStudio-集成式Prompt编辑应用.md

模块说明：PromptStudio-模块功能说明.md

本地运行（30 秒）

```
cd "D:\pp studio"
npm install
npm run dev    # http://localhost:3000
npm run build  # 产出 dist/，任意静态服务器可托管
```

体验建议路线（约 3 分钟）

① 打开线上链接 → 跳过引导 → 点「打开主文档」
② 看右侧实时校验报错，点问题跳转定位
③ 在对话面板输入修改要求（未配置 Key 时先到设置里接一家模型）
④ 点「应用为候选修改」→ 在 Diff 中逐块接受/拒绝
⑤ 打开修改日志看 Why/How/Effect → 处理完毕生成新版本 → 到「历史」回滚试试